Project Design Report – Car Rental System

Alexandre Letarte

September 28 2026

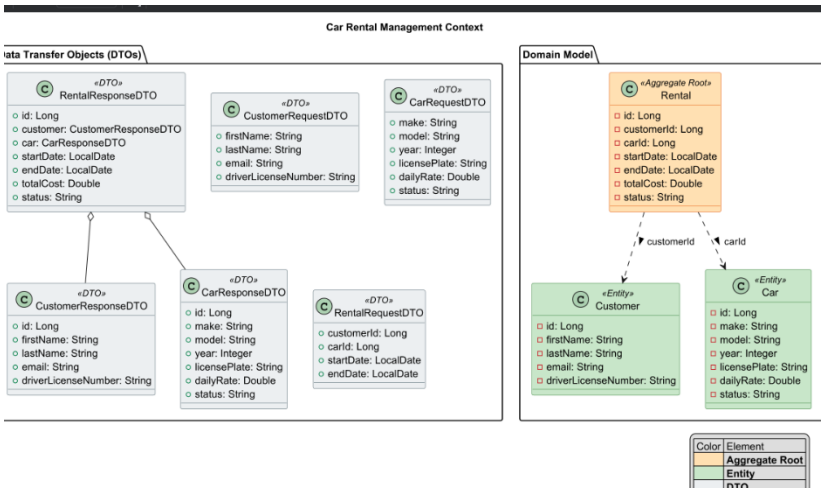
1. Introduction

The Car Rental Management System is a RESTful web service designed to manage vehicle inventory, customer profiles, and rental reservations. The system will be used by rental staff to manage available vehicles, register new customers, and handle rental bookings. Key features include maintaining customer profiles, tracking car availability and specifications, creating new rental bookings, updating rental statuses, and querying active or past rentals associated with specific customers or cars.

2. Design Section

Resources:

The system defines three primary resources (Customer, Car, Rental) along with dedicated Request and Response DTOs for each entity to ensure domain objects are not exposed directly through the API.

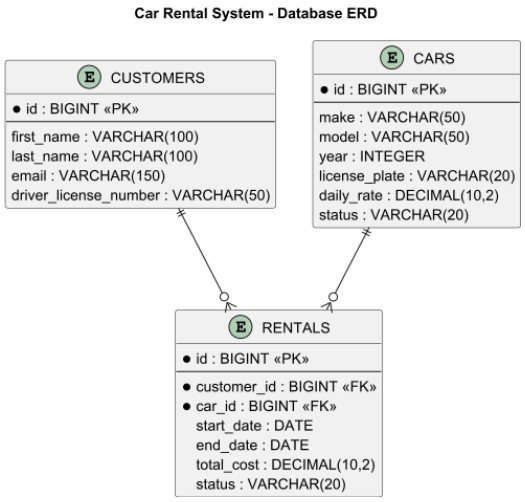


(Domain Model & DTO UML Diagram)

Relationships:

Customer to Rental: A single customer can place zero or multiple car rental bookings over time. However, a specific rental booking belongs to exactly one customer.
Car to Rental: A single car can be associated with zero or multiple distinct rental periods over its operational lifetime. Each rental booking applies to exactly one specific car.

Rental : Serves as the transactional join entity linking a Customer and a Car, storing specific transactional metadata such as start date, end date, total cost, and booking status.



(Database ERD Schema)

Fetch Strategy: The @ManyToOne relationships on Rental.customer and Rental.car use FetchType.LAZY. This ensures that querying a list of rentals does not automatically execute unnecessary SQL queries to eagerly load full customer or car records unless explicitly requested.

Cascade Strategy: No cascade deletes (CascadeType.REMOVE) is configured on Customer or Car. Deleting a customer profile or a car record will not automatically delete historical rental records, ensuring financial and audit records remain consistent in the database.

Endpoints

Customer Resource (/api/v1/customers):

GET /api/v1/customers — Retrieve all customers (Expected Status: 200 OK)

GET /api/v1/customers/{id} — Retrieve a specific customer by ID (Expected Status: 200 OK / 404 Not Found)

POST /api/v1/customers — Create a new customer profile (Expected Status: 201 Created / 400 Bad Request)

PUT /api/v1/customers/{id} — Update an existing customer profile (Expected Status: 200 OK / 400 Bad Request / 404 Not Found)

DELETE /api/v1/customers/{id} — Delete a customer profile (Expected Status: 204 No Content / 404 Not Found)

Car Resource (/api/v1/cars):

GET /api/v1/cars — Retrieve all cars in fleet (Expected Status: 200 OK)

GET /api/v1/cars/{id} — Retrieve a specific car by ID (Expected Status: 200 OK / 404 Not Found)

POST /api/v1/cars — Add a new car to inventory (Expected Status: 201 Created / 400 Bad Request)

PUT /api/v1/cars/{id} — Update car details (Expected Status: 200 OK / 400 Bad Request / 404 Not Found)

DELETE /api/v1/cars/{id} — Remove a car from inventory (Expected Status: 204 No Content / 404 Not Found)

Rental Resource (/api/v1/rentals) :

GET /api/v1/rentals — Retrieve all rental bookings (Expected Status: 200 OK)

GET /api/v1/rentals/{id} — Retrieve rental details by ID (Expected Status: 200 OK / 404 Not Found)

POST /api/v1/rentals — Create a new rental booking (Expected Status: 201 Created / 400 Bad Request / 404 Not Found)

PUT /api/v1/rentals/{id} — Update a rental booking (Expected Status: 200 OK / 400 Bad Request / 404 Not Found)

DELETE /api/v1/rentals/{id} — Cancel or delete a rental booking (Expected Status: 204 No Content / 404 Not Found)

Aggregated / Relationship Endpoints :

GET /api/v1/customers/{id}/rentals — Fetch all rental bookings placed by a specific customer (Expected Status: 200 OK / 404 Not Found)

GET /api/v1/cars/{id}/rentals — Fetch the full rental history for a specific car (Expected Status: 200 OK / 404 Not Found)

Example JSON Request & Response Payloads

Creating a New Customer (POST /api/v1/customers)

- Request Body (CustomerRequestDTO):

JSON

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "driverLicenseNumber": "D1234-56789-01"
}
```

- Response Body (201 Created - CustomerResponseDTO):

JSON

```
{
  "id": 1,
  "firstName": "John",
```

```
 "lastName": "Doe",
 "email": "john.doe@example.com",
 "driverLicenseNumber": "D1234-56789-01"
}
```


Creating a New Rental Booking (POST /api/v1/rentals)

- Request Body (RentalRequestDTO):

JSON


```
{
 "customerId": 1,
 "carId": 2,
 "startDate": "2026-10-01",
 "endDate": "2026-10-05"
}
```


- Response Body (201 Created - RentalResponseDTO):

JSON


```
{
 "id": 10,
 "customer": {
  "id": 1,
  "firstName": "John",
  "lastName": "Doe",
  "email": "john.doe@example.com",
  "driverLicenseNumber": "D1234-56789-01"
 },
 "car": {
```

```
  "id": 2,
  "make": "Toyota",
  "model": "Camry",
  "year": 2024,
  "licensePlate": "ABC-123",
  "dailyRate": 55.00,
  "status": "RENTED"
 },
 "startDate": "2026-10-01",
 "endDate": "2026-10-05",
 "totalCost": 220.00,
 "status": "BOOKED"
}
```


DTO Validation Annotations

CustomerRequestDTO:

- firstName: @NotBlank(message = "First name is required")
- lastName: @NotBlank(message = "Last name is required")
- email: @NotBlank(message = "Email is required"), @Email(message = "Invalid email format")
- driverLicenseNumber: @NotBlank(message = "Driver license number is required")

CarRequestDTO:

- make: @NotBlank(message = "Make is required")
- model: @NotBlank(message = "Model is required")

- year: @NotNull(message = "Year is required"), @Min(value = 1900, message = "Year must be valid")
- licensePlate: @NotBlank(message = "License plate is required")
- dailyRate: @NotNull(message = "Daily rate is required"), @Positive(message = "Daily rate must be greater than zero")

RentalRequestDTO:

- customerId: @NotNull(message = "Customer ID is required")
- carId: @NotNull(message = "Car ID is required")
- startDate: @NotNull(message = "Start date is required"), @FutureOrPresent(message = "Start date cannot be in the past")
- endDate: @NotNull(message = "End date is required"), @Future(message = "End date must be in the future")

Expected 400 Bad Request Error Response :

When invalid payload data is submitted to any endpoint (e.g., POST /api/v1/rentals with a missing customerId or past startDate), Spring Boot automatically catches the @Valid violation via MethodArgumentNotValidException and returns an HTTP 400 Bad Request response formatted as follows:

JSON


```
{
 "timestamp": "2026-09-27T19:46:23",
 "status": 400,
 "error": "Bad Request",
 "path": "/api/v1/rentals",
 "errors": {
  "customerId": "Customer ID is required",
  "startDate": "Start date cannot be in the past"
 }
}
```

Non-Functional Considerations

Error Handling Approach:

The application employs a centralized global exception handling mechanism using Spring's @RestControllerAdvice (GlobalExceptionHandler) to ensure consistent REST error responses across all endpoints:

- **NotFoundException**: A custom runtime exception thrown whenever a requested resource (Customer, Car, or Rental) is not found in the database. Caught globally to return an HTTP 404 Not Found status with a descriptive JSON error payload.
- **MethodArgumentNotValidException**: Automatically triggered when request payload validation fails via @Valid. Caught globally to map validation field errors into a structured HTTP 400 Bad Request response.
- **IllegalArgumentException / General Exceptions**: Caught to handle invalid business logic operations (e.g., attempting to book a car that is already rented) and returned as an HTTP 400 Bad Request or 500 Internal Server Error.

Naming & Packaging Architecture

The project follows a standard 3-layer architecture with a clear separation of concerns using a layered package structure:

com.champlain.carrental

├controller/ # REST Controllers handling HTTP requests & mapping endpoints

├ service/ # Business logic layer handling domain rules & validation

├repository/ # Spring Data JPA repositories interfacing with H2 database

├entity/ # Persistent JPA domain entities (@Entity)

├ dto/ # Request and Response DTOs

└── exception/ # Custom exceptions & @RestControllerAdvice exception handler

Naming Conventions: REST endpoints use plural lowercase nouns (/api/v1/customers, /api/v1/cars, /api/v1/rentals). DTOs and Entities use PascalCase (CustomerRequestDTO, Rental). Java methods and variable names follow camelCase conventions.

Testing Approach

A comprehensive testing strategy will be executed in subsequent milestones using JUnit 5, Mockito, and Spring Boot Test framework:

- **Unit Tests**: Mockito will be used to isolate and unit-test the service layer logic (e.g., verifying rental cost calculation algorithms and handling of missing entities).
- **Integration Tests**: @WebMvcTest and @DataJpaTest will be used to test REST controller endpoint responses, HTTP status code mappings, JSON serialization, and Spring Data JPA repository database operations against the in-memory H2 database.